Algorithm comparison

give me a comparison between Merge Sort, Quick Sort, Count Sort, Heap Sort algorithms

Here's a clear **comparison between Merge Sort, Quick Sort, Count Sort, and Heap Sort** across multiple aspects:

🔹 1. Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$
Count Sort	O(n + k)	O(n + k)	O(n + k)
Heap Sort	O(n log n)	O(n log n)	O(n log n)

- Here,
  - `n` = number of elements
  - `k` = range of input values (max element – min element)

🔹 2. Space Complexity

Algorithm	Space Complexity
Merge Sort	O(n) (extra arrays for merging)
Quick Sort	O(log n) (recursive stack, in-place)
Count Sort	O(k) (extra array for counts)
Heap Sort	O(1) (in-place)

🔹 3. Stability

- **Merge Sort** → ✅ Stable (preserves order of equal elements)
- **Quick Sort** → ❌ Not stable (unless modified)
- **Count Sort** → ✅ Stable (when implemented properly)
- **Heap Sort** → ❌ Not stable

🔹 4. Nature of Algorithm

Algorithm	Type
Merge Sort	Divide and Conquer
Quick Sort	Divide and Conquer
Count Sort	Non-comparison, Distribution-based
Heap Sort	Comparison-based, Heap Data Structure

🔹 5. When to Use

- **Merge Sort** → Good for linked lists, external sorting (large data on disk), when stability is required.
- **Quick Sort** → Best for general-purpose sorting in memory, average case is very fast, but avoid for already sorted data without randomization.
- **Count Sort** → Best for integers or small range of keys (`k` not much larger than `n`). Not suitable for large range values.
- **Heap Sort** → Useful when constant space is important. Slightly slower in practice than Quick Sort due to cache inefficiency.

🔹 6. Practical Notes

- **Merge Sort**: Consistent O(n log n) but requires extra memory.
- **Quick Sort**: Faster in practice (good cache performance), but worst-case $O(n^2)$. Usually fixed by randomization (Randomized Quick Sort).

- **Count Sort**: Only works efficiently for integers or discrete values with limited range. Not comparison-based.
- **Heap Sort**: Memory efficient, but slower in practice than Quick Sort.

✅ **Summary**:

- Use **Merge Sort** when stability is important or for linked lists.
- Use **Quick Sort** for general-purpose fast sorting (with randomization).
- Use **Count Sort** for integers with a small range.
- Use **Heap Sort** when memory is tight and you want guaranteed O(n log n).

Do you want me to also make a **visual comparison table (like a cheat sheet chart)** so you can quickly review it for exams?